PROGRAMACIÓN DIDÁCTICA: PROGRAMACIÓN (0485)

Resumen de Unidades de Programación (Curso 2026-2027)

Código	Título de la UP
UP01	**INTRODUCCION A LA PROGRAMACIÓN Y JAVA**
UP02	**PENSAMIENTO COMPUTACIONAL**
UP03	**ESTRUCTURAS DE PROGRAMACIÓN**
UP04	**PROGRAMACIÓN MODULAR**
UP05	**ESTRUCTURAS DE DATOS**
UP06	**PROGRAMACIÓN ORIENTADA A OBJETOS**
UP07	**USO AVANZADO DE CLASES**
UP08	**GESTIÓN DE FICHEROS**
UP09	**ACCESO A DATOS**
UP10	**INTERFAZ GRÁFICA DE USUARIO**

- La suma total de estas unidades es de **266 horas**, correspondientes íntegramente al periodo de formación realizado en el centro educativo fuera del periodo dual.

UP01: INTRODUCCION A LA PROGRAMACIÓN Y JAVA

1. Identificación

Campo	Detalle
Código	UP01
Módulo	Programación (0485)
Duración	**24 Horas**
Temporalización	Del **10/09/2026** al **30/09/2026**

2. Fundamentación

Resultado de Aprendizaje * **RA01.** Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

Criterios de Evaluación * **a)** Se han identificado los bloques que componen la estructura de un programa informático. * **b)** Se han creado proyectos de desarrollo de aplicaciones. * **c)** Se han utilizado entornos integrados de desarrollo. * **d)** Se han identificado los distintos tipos de variables y la utilidad específica de cada uno. * **e)** Se ha modificado el código de un programa para crear y utilizar variables. * **f)** Se han creado y utilizado constantes y literales. * **g)** Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje. * **h)** Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas. * **i)** Se han introducido comentarios en el código.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Introducción a la programación: algoritmos, programas y lenguajes de programación. * Clasificación de los lenguajes: alto/bajo nivel, interpretados/compilados, paradigmas. * Entornos de desarrollo integrados (IDE): elementos y uso del IDE (IntelliJ IDEA / VS Code). * El JDK: JVM, JRE, compilador `javac` y empaquetador `jar`. * Proyectos Java: estructura de carpetas, paquetes (`package`) e `import`. * Variables: declaración, tipos primitivos (`int`, `double`, `boolean`, `char`) y `String`. * Literales y constantes (`final`). * Operadores: aritméticos, relacionales y lógicos. Expresiones. * Conversiones de tipo: automáticas y *casting* explícito. * Comentarios: de línea (`//`), de bloque (`/* */`) y JavaDoc (`/** */`). * Funciones de librería de las clases `Math` y `String`. * Normas y estilo de codificación: nomenclatura, indentación, separaciones y organización de ficheros. * Flujos de entrada y salida: concepto de *stream*, `System.in` y `System.out`. * Entrada desde teclado con `Scanner`: `nextInt()`, `nextDouble()`, `nextLine()`. * Salida por pantalla: `print()`, `println()`, `printf()` y `DecimalFormat`.

Metodología * Metodologías activas centradas en el alumnado. El aprendizaje se organiza en torno a **prácticas guiadas de programación** donde el estudiante construye, de forma incremental, pequeños programas Java usando el IDE, descubriendo los conceptos a medida que los necesita.

Secuencia de actividades * **A1:** Introducción a la programación y al IDE. Creación del primer proyecto Java (“Hola Mundo”). Identificación de la estructura básica de un programa. * **A2:** Variables y tipos de datos. Declaración, asignación y uso de variables de diferentes tipos en un programa sencillo. * **A3:** Constantes, literales y operadores. Escritura de expresiones con operadores aritméticos, relacionales y lógicos. * **A4:** Conversiones de tipo (automáticas y *casting*) y comentarios JavaDoc. Documentación del código desarrollado. * **A5:** Flujos de E/S. Lectura de datos desde teclado y salida formateada por pantalla.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA01. * **Rúbricas de prácticas (20%):** valoración sistemática de los programas desarrollados en cada actividad, atendiendo a la corrección técnica, el estilo de codificación y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP02: PENSAMIENTO COMPUTACIONAL

1. Identificación

Campo	Detalle
Código	UP02
Módulo	Programación (0485)
Duración	**11 Horas**
Temporalización	Del **01/10/2026** al **08/10/2026**

2. Fundamentación

Resultado de Aprendizaje * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Criterios a, b, c)*

Criterios de Evaluación * **a)** Se ha escrito y probado código que haga uso de estructuras de selección. * **b)** Se han utilizado estructuras de repetición. * **c)** Se han reconocido las posibilidades de las sentencias de salto.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos

- Programación estructurada: estructuras secuencial, de selección y repetitiva.
- Bucles anidados y bucles infinitos.
- Introducción a los algoritmos: representación mediante diagramas de flujo y pseudocódigo.

Metodología * Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado resuelve problemas de dificultad creciente implementando y depurando programas con estructuras de control.

Secuencia de actividades

- **A1:** Estructuras de selección. Programas de clasificación y toma de decisiones en diagramas de flujo y pseudocódigo.
- **A2:** Estructuras de repeticiónen diagramas de flujo y pseudocódigo. Bucles simples y anidados.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03 trabajados en esta UP. * **Rúbricas de prácticas (20%):** valoración de los programas desarrollados en cada actividad, atendiendo a la corrección lógica, la elección de la estructura de control adecuada y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP03: ESTRUCTURAS DE PROGRAMACIÓN

1. Identificación

Campo	Detalle
Código	UP03

Campo	Detalle
Módulo	Programación (0485)
Duración	**24 Horas**
Temporalización	Del **13/10/2026** al **29/10/2026**

2. Fundamentación

Resultado de Aprendizaje * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Criterios a, b, c, e, f, g — la gestión de excepciones se completa en UP06)*

Criterios de Evaluación * **a)** Se ha escrito y probado código que haga uso de estructuras de selección. * **b)** Se han utilizado estructuras de repetición. * **c)** Se han reconocido las posibilidades de las sentencias de salto. * **e)** Se han creado programas ejecutables utilizando diferentes estructuras de control. * **f)** Se han probado y depurado los programas. * **g)** Se ha comentado y documentado el código.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos

- Programación estructurada: estructuras secuencial, de selección y repetitiva.
- Estructuras de selección: `if`, `if-else`, operador ternario `? :`, `if-else if-else` y `switch`.
- Estructuras de repetición: `while`, `do-while`, `for` y `for-each`.
- Bucles anidados y bucles infinitos.
- Estructuras de salto: `break`, `continue` y `return`.
- Depuración de programas con el depurador del IDE: *breakpoints*, ejecución paso a paso y visualización de variables.

Metodología * Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado resuelve problemas de dificultad creciente implementando y depurando programas con estructuras de control.

Secuencia de actividades

- **A1:** Estructuras de selección (`if`, `if-else`, `switch`). Programas de clasificación y toma de decisiones.

- **A2:** Estructuras de repetición (`while`, `do-while`, `for`). Bucles simples y anidados.
- **A3:** Sentencias de salto (`break`, `continue`) y depuración con el IDE. Algoritmos: diagramas de flujo y pseudocódigo.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03 trabajados en esta UP. * **Rúbricas de prácticas (20%):** valoración de los programas desarrollados en cada actividad, atendiendo a la corrección lógica, la elección de la estructura de control adecuada y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP04: PROGRAMACIÓN MODULAR

1. Identificación

Campo	Detalle
Código	UP04
Módulo	Programación (0485)
Duración	**24 Horas**
Temporalización	Del **03/11/2026** al **19/11/2026**

2. Fundamentación

Resultados de Aprendizaje

- **RA02.** Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

- **RA04.** Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.

Criterios de Evaluación * **RA02. d)** Se han utilizado métodos y propiedades de los objetos. * **RA02. e)** Se han escrito llamadas a métodos estáticos. * **RA02. f)** Se han utilizado parámetros en la llamada a métodos. * **RA02. g)** Se han incorporado y utilizado librerías de objetos. * **RA04. c)** Se han definido propiedades y métodos. * **RA04. e)** Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente. * **RA04. h)** Se han creado y utilizado métodos estáticos. * **RA04. i)** Se han creado y utilizado conjuntos y librerías de clases.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Concepto de programación modular: división de un programa en partes independientes y reutilizables. * Ventajas de la programación modular: reutilización, organización, mantenimiento, legibilidad y reducción de la complejidad. * Procedimientos y funciones: concepto, finalidad y estructura. * Definición de métodos en Java: modificadores, tipo de retorno, nombre y parámetros. * Métodos `void` y métodos con valor de retorno. * Parámetros y argumentos: declaración, paso de información y utilización dentro de los métodos. * Valores de retorno mediante la sentencia `return`. * Llamada a métodos y paso de argumentos. * Variables locales y ámbito de las variables. * Métodos estáticos y métodos de instancia. * Utilización de métodos desde una misma clase. * Utilización de métodos definidos en otras clases. * Creación de clases destinadas a agrupar métodos relacionados y reutilizables. * Organización del código en diferentes clases y paquetes. * Reutilización de métodos mediante la creación y utilización de librerías de clases. * Sobrecarga de métodos: definición de varios métodos con el mismo nombre y diferentes parámetros. * Documentación de métodos mediante comentarios y Javadoc. * Diseño de programas mediante la descomposición de un problema en métodos y clases independientes.

Metodología

- Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado parte de programas sencillos y aprende a dividirlos en métodos independientes, reutilizables y fáciles de mantener. Posteriormente, organiza estos métodos en diferentes clases y los utiliza desde otras clases, aplicando los principios básicos de la programación modular y la Programación Orientada a Objetos.

Secuencia de actividades * **A1:** Introducción a la programación modular: identificación de problemas susceptibles de dividirse en partes independientes y ventajas de la modularización. * **A2:** Creación y utilización de métodos en Java: métodos `void`, métodos con retorno, parámetros, argumentos

y sentencia `return`. * **A3:** Ámbito de las variables y métodos estáticos. Utilización de diferentes métodos desde una misma clase para resolver problemas de programación. * **A4:** Creación de clases auxiliares y utilización de métodos desde otras clases. Organización del código en clases y paquetes. * **A5:** Sobrecarga de métodos, reutilización de código y creación de pequeñas librerías de clases. Documentación de los métodos mediante Javadoc. * **A6:** Desarrollo de una aplicación completa aplicando programación modular: descomposición del problema, creación de métodos y clases, reutilización de código y documentación.

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03 trabajados en esta UP. * **Rúbricas de prácticas (20%):** valoración de los programas desarrollados en cada actividad, atendiendo a la corrección lógica, la elección de la estructura de control adecuada y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP05: ESTRUCTURAS DE DATOS

1. Identificación

Campo	Detalle
Código	UP05
Módulo	Programación (0485)
Duración	**29 Horas**
Temporalización	Del **24/11/2026** al **17/12/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA06.** Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.

Criterios de Evaluación * **a)** Se han escrito programas que utilicen matrices (arrays). * **b)** Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados. * **c)** Se han utilizado listas para almacenar y procesar información. * **d)** Se han utilizado iteradores para recorrer los elementos de las listas. * **e)** Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles. * **j)** Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.

Competencias

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Concepto de estructura de datos: organización y almacenamiento de información para facilitar su acceso y manipulación. * Arrays: características, declaración, creación e inicialización. * Acceso a los elementos de un array mediante índices. * Recorrido y modificación de los elementos de un array. * Arrays multidimensionales y representación de tablas de datos. * Limitaciones de los arrays: tamaño fijo y necesidad de gestionar manualmente determinadas operaciones. * Introducción a las estructuras de datos dinámicas y a las colecciones de Java. * `ArrayList`: características, declaración, creación y principales operaciones. * Inserción, consulta, modificación y eliminación de elementos en un `ArrayList`. * Recorrido de un `ArrayList` mediante bucles e iteradores. * Comparación entre arrays y `ArrayList`: características, ventajas y limitaciones. * Concepto de pila (*stack*): estructura LIFO (*Last In, First Out*). * Operaciones básicas de una pila: inserción (`push`), consulta del elemento superior (`peek`) y extracción (`pop`). * Implementación de pilas mediante las clases de la API de colecciones de Java. * Concepto de cola (*queue*): estructura FIFO (*First In, First Out*). * Operaciones básicas de una cola: inserción (`offer`/`add`), consulta del primer elemento (`peek`) y extracción (`poll`/`remove`). * Implementación de colas mediante las clases e interfaces de la API de colecciones de Java. * Comparación entre arrays, `ArrayList`, pilas y colas según sus características y operaciones disponibles. * Selección de la estructura de datos más adecuada según las necesidades de una aplicación.

Metodología * Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado trabaja con diferentes estructuras de datos para almacenar, consultar, modificar y eliminar información. Se comienza con arrays y se evoluciona hacia estructuras dinámicas y especializadas como `ArrayList`, pilas y colas, comparando sus características y seleccionando la estructura más adecuada para cada problema.

Secuencia de actividades * **A1:** Introducción a las estructuras de datos. Creación, inicialización, acceso, modificación y recorrido de arrays. * **A2:** Arrays multidimensionales y resolución de problemas mediante tablas de datos. * **A3:** Introducción a las colecciones de Java. Creación y utilización de

`ArrayList`: inserción, consulta, modificación, eliminación y recorrido de elementos. * **A4:** Pilas: concepto LIFO y operaciones `push`, `pop` y `peek`. Resolución de problemas mediante estructuras de tipo pila. * **A5:** Colas: concepto FIFO y operaciones de inserción, consulta y extracción. Resolución de problemas mediante estructuras de tipo cola. * **A6:** Comparación y selección de estructuras de datos. Desarrollo de una aplicación que utilice arrays, `ArrayList`, pilas y colas según las necesidades de cada parte del problema.

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03 trabajados en esta UP. * **Rúbricas de prácticas (20%):** valoración de los programas desarrollados en cada actividad, atendiendo a la corrección lógica, la elección de la estructura de control adecuada y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP06: PROGRAMACIÓN ORIENTADA A OBJETOS

1. Identificación

Campo	Detalle
Código	UP06
Módulo	Programación (0485)
Duración	**24 Horas**
Temporalización	Del **22/12/2026** al **22/01/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA02.** Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos. * **RA04.** Desarrolla programas organizados

en clases analizando y aplicando los principios de la programación orientada a objetos. *(Criterios a–e, inicio)*

Criterios de Evaluación * **RA02. a)** Se han identificado los fundamentos de la programación orientada a objetos. * **RA02. b)** Se han escrito programas simples. * **RA02. c)** Se han instanciado objetos a partir de clases predefinidas. * **RA02. d)** Se han utilizado métodos y propiedades de los objetos. * **RA02. e)** Se han escrito llamadas a métodos estáticos. * **RA02. f)** Se han utilizado parámetros en la llamada a métodos. * **RA02. g)** Se han incorporado y utilizado librerías de objetos. * **RA02. h)** Se han utilizado constructores. * **RA02. i)** Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples. * **RA04. a)** Se ha reconocido la sintaxis, estructura y componentes típicos de una clase. * **RA04. b)** Se han definido clases. * **RA04. c)** Se han definido propiedades y métodos. * **RA04. d)** Se han creado constructores. * **RA04. e)** Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Introducción a la POO: abstracción, clases y objetos. * Atributos de instancia y de clase (`static`). Métodos de instancia y estáticos. * Constructores: constructor por defecto y sobrecarga de constructores. * Encapsulamiento y modificadores de acceso (`public`, `private`, `protected`). * *Getters* y *setters*. * La palabra clave `this`. * Uso de librerías y clases predefinidas de Java (`Math`, `String`, `Integer`, etc.). * Relaciones entre clases: asociación, agregación y composición. * Normas de codificación en POO: nomenclatura de clases, métodos y atributos.

Metodología * Metodologías activas basadas en **prácticas de programación** orientadas a objetos. El alumnado modela objetos del mundo real, define sus clases con atributos y métodos, crea instancias y las utiliza en programas completos.

Secuencia de actividades * **A1:** Introducción a la POO. Abstracción y modelado de clases sencillas con atributos y métodos. Creación del primer objeto. * **A2:** Constructores (por defecto y sobrecargados). Encapsulamiento, *getters* y *setters*. Uso de `this`. * **A3:** Métodos estáticos. Uso de librerías predefinidas (`Math`, `String`). Llamadas a métodos con parámetros. * **A4:** Relaciones entre clases (asociación, agregación, composición). Diseño y desarrollo de un programa con varias clases relacionadas.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA02 y del inicio del RA04. * **Rúbricas de prácticas (20%):** valoración de los

programas orientados a objetos desarrollados en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP07: USO AVANZADO DE CLASES

1. Identificación

Campo	Detalle
Código	UP07
Módulo	Programación (0485)
Duración	**32 Horas**
Temporalización	Del **26/01/2026** al **18/02/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA04.** Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos. *(Criterios f–i, continuación)* * **RA06.** Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos. *(Criterios a–e)*

Criterios de Evaluación * **RA04. f)** Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros. * **RA04. g)** Se han definido y utilizado clases heredadas. * **RA04. h)** Se han creado y utilizado métodos estáticos. * **RA04. i)** Se han creado y utilizado conjuntos y librerías de clases. * **RA06. a)** Se han escrito programas que utilicen matrices (*arrays*). * **RA06. b)** Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados. * **RA06. c)** Se han utilizado listas para almacenar y procesar información. * **RA06. d)** Se han utilizado iteradores para recorrer los elementos de las listas. * **RA06. e)** Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Herencia: superclases y subclases, la palabra clave `extends`. Transititividad. * Modificadores de visibilidad en jerarquías de clases. * Herencia de constructores: `super()` y `this()`. * Sobreescritura de métodos (`@Override`) y sobreescritura del método `toString()`. * Clases abstractas y métodos abstractos. Clases finales y métodos finales. * Interfaces: definición, implementación (`implements`) y diferencias con clases abstractas. * Polimorfismo: sobrecarga y sobreescritura. Polimorfismo en colecciones. * *Arrays* unidimensionales y multidimensionales: declaración, inicialización y recorrido. * Colecciones Java (`java.util`): `ArrayList`, `LinkedList`, `HashMap`. Iteradores. * Características y ventajas de cada tipo de colección. Selección según caso de uso.

Metodología * Metodologías activas basadas en **prácticas de programación**. El alumnado diseña jerarquías de clases, implementa interfaces y desarrolla programas que emplean colecciones para gestionar conjuntos de objetos.

Secuencia de actividades * **A1:** Herencia simple. Diseño y codificación de jerarquías de clases. Constructores heredados (`super`). * **A2:** Sobreescritura de métodos (`@Override`). Clases abstractas e interfaces. Polimorfismo. * **A3:** *Arrays* unidimensionales y multidimensionales. Recorrido y operaciones básicas. * **A4:** Colecciones (`ArrayList`, `HashMap`). Iteradores. Selección de la colección adecuada según el problema.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA04 (continuación) y del RA06 (parcial). * **Rúbricas de prácticas (20%):** valoración de los programas con jerarquías de clases y colecciones desarrollados en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP08: GESTION DE FICHEROS

1. Identificación

Campo	Detalle
Código	UP08
Módulo	Programación (0485)
Duración	**16 Horas**
Temporalización	Del **23/02/2027** al **04/03/2027**

2. Fundamentación

Resultados de Aprendizaje * **RA05.** Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases. * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Criterios d, h, i — gestión de excepciones y aserciones)*

Criterios de Evaluación * **RA05. a)** Se ha utilizado la consola para realizar operaciones de entrada y salida de información. * **RA05. b)** Se han aplicado formatos en la visualización de la información. * **RA05. c)** Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas. * **RA05. d)** Se han utilizado ficheros para almacenar y recuperar información. * **RA05. e)** Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros. * **RA05. f)** Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples. * **RA05. g)** Se han programado controladores de eventos. * **RA05. h)** Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información. * **RA03. d)** Se ha escrito código utilizando control de excepciones. * **RA03. h)** Se han creado excepciones. * **RA03. i)** Se han utilizado aserciones para la detección y corrección de errores durante la fase de desarrollo.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Concepto de flujo (*stream*): flujos de bytes y de caracteres. Jerarquía `java.io`. * Flujos estándar: `System.in`, `System.out`, `System.err`. * Entrada desde teclado: `Scanner` y sus métodos de lectura. * Salida por pantalla con formato: `printf`, `DecimalFormat`. * Sistema de ficheros: clase `File`. Crear, borrar, renombrar y listar ficheros y directorios. * Lectura y escritura de ficheros de texto: `FileReader`, `BufferedReader`, `FileWriter`, `PrintWriter`. * Lectura y escritura de ficheros binarios: `FileInputStream`, `FileOutputStream`. * Persistencia y serialización: `ObjectOutputStream`, `ObjectInputStream`, interfaz `Serializable`. * Gestión de excepciones: jerarquía `Throwable`. Excepciones *checked* y *unchecked*. * Captura de excepciones:

`try-catch-finally`. Excepciones múltiples. * Propagación de excepciones: `throws` y `throw`. * Creación de excepciones propias (subclases de `Exception`). * Aserciones (`assert`) para la detección de errores en fase de desarrollo. * Interfaces gráficas de usuario (GUI) con Java Swing: ventanas, componentes básicos y *layout*. * Controladores de eventos: *listeners* y manejo de eventos de usuario.

Metodología * Metodologías activas basadas en **prácticas de programación**. El alumnado desarrolla aplicaciones que leen y escriben datos desde distintas fuentes (consola, ficheros), gestiona los errores mediante excepciones y construye interfaces gráficas sencillas.

Secuencia de actividades * **A1:** Flujos de E/S. Lectura y escritura de ficheros de texto y binarios. Sistema de ficheros con `File`. * **A2:** Persistencia y serialización de objetos. Lectura y escritura de objetos en ficheros. * **A3:** Gestión de excepciones: captura (`try-catch-finally`) y propagación (`throws`/`throw`). Creación de excepciones propias. Aserciones. * **A4:** Interfaces gráficas con Java Swing: ventanas, botones, campos de texto y cuadros de diálogo. Controladores de eventos. Programa completo con GUI de entrada y salida.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA05 y de los criterios restantes del RA03. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de E/S, ficheros y GUI desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP09: ACCESO A DATOS

1. Identificación

Campo	Detalle
Código	UP07
Módulo	Programación (0485)

Campo	Detalle
Duración	**21 Horas**
Temporalización	Del **09/03/2027** al **08/04/2027**

2. Fundamentación

Resultados de Aprendizaje * **RA08.** Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información. * **RA09.** Gestiona información almacenada en bases de datos manteniendo la integridad y consistencia de los datos.

Criterios de Evaluación * **RA08. a)** Se han identificado las características de las bases de datos orientadas a objetos. * **RA08. b)** Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos. * **RA08. c)** Se han instalado sistemas gestores de bases de datos orientados a objetos. * **RA08. d)** Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada. * **RA08. e)** Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos. * **RA08. f)** Se han programado aplicaciones que almacenen objetos en las bases de datos creadas. * **RA08. g)** Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos. * **RA08. h)** Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados. * **RA09. a)** Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos. * **RA09. b)** Se han programado conexiones con bases de datos. * **RA09. c)** Se ha escrito un código para almacenar información en bases de datos. * **RA09. d)** Se han creado programas para recuperar y mostrar información almacenada en bases de datos. * **RA09. e)** Se han efectuado borrados y modificaciones sobre la información almacenada. * **RA09. f)** Se han creado aplicaciones que muestren la información almacenada en bases de datos. * **RA09. g)** Se han creado aplicaciones para gestionar la información presente en bases de datos.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos

Bases de datos orientadas a objetos (RA08): * Características de las BBDD orientadas a objetos frente a las relacionales. * Sistemas gestores de BBDD orientadas a objetos: instalación y configuración. * Creación de bases de datos y estructuras de almacenamiento de objetos. * Operaciones CRUD sobre

objetos: almacenamiento, recuperación, actualización y eliminación. * Almacenamiento de tipos de datos estructurados, compuestos y relacionados.

Acceso a bases de datos relacionales con JDBC (RA09): * Repaso del modelo relacional y SQL: DDL, DML y TCL. * El driver JDBC: tipos de drivers, instalación y configuración. * Establecimiento y cierre de la conexión (`Connection`, `DriverManager`). * Ejecución de sentencias SQL: `Statement` y `PreparedStatement`. * Recuperación de resultados: objeto `ResultSet` y sus métodos de navegación. * Operaciones CRUD desde Java: INSERT, SELECT, UPDATE y DELETE. * Gestión de transacciones: propiedades ACID, `commit()` y `rollback()`. * Aplicaciones completas con interfaz de usuario que gestionan información en BBDD.

Metodología * Metodologías activas basadas en **prácticas de programación** y un proyecto integrador final. El alumnado desarrolla aplicaciones Java que persisten objetos en bases de datos, tanto orientadas a objetos como relacionales, gestionando el acceso mediante JDBC.

Secuencia de actividades * **A1:** Instalación y configuración del SGBD orientado a objetos. Creación de estructuras de almacenamiento y operaciones CRUD sobre objetos. * **A2:** Tipos de datos estructurados y relacionados en BBDD orientadas a objetos. Programas de almacenamiento y recuperación. * **A3:** Driver JDBC. Conexión a una BBDD relacional (MySQL). Ejecución de sentencias SQL con `Statement` y `PreparedStatement`. * **A4:** Recuperación de resultados (`ResultSet`). Operaciones CRUD completas. Gestión de transacciones. * **A5:** Aplicación final: programa Java con interfaz de usuario que gestiona información almacenada en una base de datos relacional.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK, SGBD orientado a objetos, servidor MySQL, driver JDBC (`mysql-connector-java`), control de versiones (Git/GitHub) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA08 y del RA09. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de acceso a bases de datos desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar cada RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas.

UP10: INTERFAZ GRÁFICA DE USUARIO

1. Identificación

Campo	Detalle
Código	UP10
Módulo	Programación (0485)
Duración	**40 Horas**
Temporalización	Del **13/04/2027** al **13/05/2027**

2. Fundamentación

Resultados de Aprendizaje * **RA05.** Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases. * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Contribución relacionada con la gestión de eventos y el control de errores en las interfaces gráficas)*

Criterios de Evaluación * **RA05. f)** Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples. * **RA05. g)** Se han programado controladores de eventos. * **RA05. h)** Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información. * **RA03. f)** Se han probado y depurado los programas. * **RA03. g)** Se ha comentado y documentado el código. * **RA03. d)** Se ha escrito código utilizando control de excepciones. *(Aplicación a la gestión de errores en las interfaces gráficas)*

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos

- Concepto de interfaz gráfica de usuario (GUI): características, finalidad y ventajas frente a las interfaces basadas en consola.
- Componentes de una interfaz gráfica: ventanas, botones, etiquetas, campos de texto, áreas de texto, casillas de selección y listas.
- Creación de interfaces gráficas con Java Swing.
- La clase `JFrame`: creación y configuración de ventanas.

- Componentes básicos de Swing: `JLabel`, `JButton`, `JTextField`, `JTextArea`, `JCheckBox`, `JRadioButton` y `JComboBox`.
- Contenedores y organización de componentes.
- Gestores de distribución (*layout managers*): `FlowLayout`, `BorderLayout`, `GridLayout` y `GridBagLayout`.
- Propiedades y configuración de los componentes: tamaño, posición, texto, visibilidad y estado.
- Eventos y programación dirigida por eventos.
- Controladores de eventos mediante *listeners*.
- Gestión de eventos de botones mediante `ActionListener`.
- Gestión de eventos asociados al teclado, ratón y otros componentes.
- Obtención y modificación de los datos introducidos por el usuario.
- Cuadros de diálogo mediante `JOptionPane`.
- Validación de los datos introducidos en los componentes gráficos.
- Gestión de errores y excepciones en aplicaciones con interfaz gráfica.
- Organización del código de una aplicación gráfica en diferentes clases.
- Desarrollo de aplicaciones completas con interfaces gráficas de usuario.

Metodología

- Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado comienza creando ventanas y componentes gráficos sencillos y evoluciona hacia aplicaciones interactivas completas. Se trabaja especialmente la programación dirigida por eventos, haciendo que los componentes de la interfaz respondan a las acciones realizadas por el usuario. Finalmente, se integran los conocimientos adquiridos en una aplicación gráfica funcional.

Secuencia de actividades * **A1:** Introducción a las interfaces gráficas. Creación y configuración de ventanas mediante `JFrame` y utilización de los componentes básicos de Java Swing. * **A2:** Organización de los componentes mediante gestores de distribución. Creación de formularios utilizando etiquetas, campos de texto, botones, casillas de selección, listas y otros componentes. * **A3:** Programación dirigida por eventos. Utilización de `ActionListener` y otros *listeners* para responder a las acciones del usuario. * **A4:** Entrada y validación de datos mediante componentes gráficos. Uso de `JOptionPane` para mostrar información, solicitar datos y comunicar errores. * **A5:** Organización de aplicaciones gráficas en diferentes clases y gestión de excepciones y errores producidos durante la interacción con el usuario. * **A6:** Desarrollo de una aplicación completa con Java Swing que integre ventanas, componentes, gestores de distribución, eventos, validación de datos y comunicación con el usuario.

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA08 y del RA09. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de acceso a bases de datos desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar cada RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas.